

FIAP – Faculdade de Informática e Administração Paulista

DevOps Tools & Cloud Computing

Challenge 1º semestre 2º ano – Containerização em Nuvem

Projeto Integrado – Pethub

Equipe: Vitalis

Integrantes

RM	Nome Completo	Turma
RM566234	João Guilherme Carvalho Novaes	2TDSPV
RM561489	Ana Flavia Camelo	2TDSPV
RM562745	Gustavo Kenji Terada	2TDSPV
RM565154	Pedro Chasci Puga	2TDSPV
RM561342	Lucas Figueiredo Vieira	2TDSPV

Índice

1. Folha de Rosto
2. Descrição do Projeto
3. Desenho da Arquitetura Macro
4. Tecnologias Utilizadas
5. Instalação da Solução (How to)
6. Evidência de Remoção da VM
7. Link do Repositório GitHub
8. Link do Vídeo no YouTube

2. Descrição do Projeto

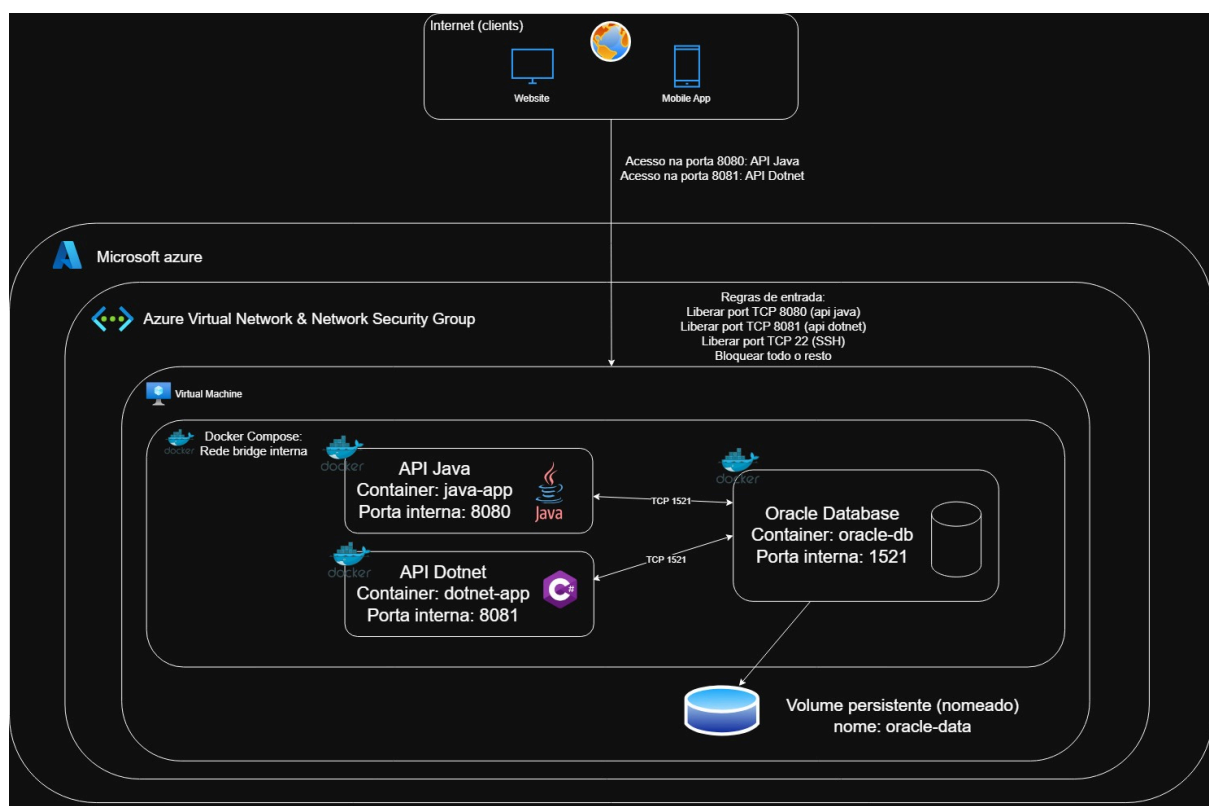
O **PetHub** é um ecossistema digital integrado projetado para revolucionar a gestão de cuidados veterinários, conectando de forma inteligente clínicas, veterinários, tutores e os próprios pets. O coração dessa arquitetura é uma API robusta desenvolvida em **Spring Boot 3.4+**, que interage com uma base de dados relacional **Oracle Database** para centralizar informações clínicas críticas e automatizar rotinas operacionais e preventivas. O sistema destaca-se pelas seguintes capacidades e domínios de negócio:

- **Prontuário e Histórico Clínico Unificado:** Gerenciamento completo de **Pets** (identificação, espécie, raça, idade e peso), **Tutores** (vinculados e integrados com microsserviços legados) e **Veterinários** (perfis profissionais com controle de CRMV e especialidades).
- **Gestão de Atendimento e Consultas:** Agendamento completo de consultas médicas integrando fluxos de notificação ativa para os tutores sobre datas e preparações necessárias.
- **Diagnósticos Clínicos e Inteligência Artificial:** Cadastro detalhado de consultas e diagnósticos clínicos. A API é estruturada para suportar modelos de Machine Learning e Inteligência Artificial Generativa, registrando sintomas específicos (como perda de apetite, tosse, alterações de temperatura e dificuldades respiratórias), gerando previsões de patologias com níveis de confiança (`doenca_predita` e `confianca_predicao`) e armazenando análises clínicas detalhadas geradas por GenAI (`analise_gen_ai`).
- **Medicina Preventiva e Lembretes (Vacinas e Tratamentos):** Controle de aplicação de vacinas, medicamentos e procedimentos terapêuticos, monitorando a

necessidade de doses futuras e integrando-se a canais de mensageria para alertar os tutores.

- **Prescrições e Pedidos Médicos:** Registro centralizado de exames solicitados e medicamentos receitados para acompanhamento em domicílio.
- **Monitoramento Contínuo via IoT (Wearables):** A plataforma está preparada para receber telemetria contínua vinda de coleiras ou de outros dispositivos vestíveis (*wearables*). A API processa e armazena leituras de temperatura corporal e frequência cardíaca dos animais em tempo real, disparando alertas automáticos caso sejam detectadas anomalias nos sinais vitais.

3. Desenho da Arquitetura Macro



4. Tecnologias Utilizadas

- Linguagem: Java 17 + Spring Boot 3.4 + C#(.NET)
- Banco de dados: Oracle XE (gvenzl/oracle-xe)
- Containerização: Docker e Docker Compose
- Cloud: Microsoft Azure (VM Linux Ubuntu 22.04)
- Automação: Azure CLI
- Versionamento: Git + GitHub

5. Instalação da Solução (How to)

O projeto foi totalmente configurado para subir através do **Docker Compose**. Siga os passos abaixo para executar a aplicação localmente ou na sua VM na Azure.

Pré-requisitos

- Git
- Docker e Docker Compose instalados.

Passos para Execução:

1 - Clone este repositório:

```
git clone https://github.com/vitalis-sa/challenge1sem2026-devops.git  
cd challenge1sem2026-devops
```

2 - Configure as Variáveis de Ambiente: Copie o arquivo de exemplo e edite as credenciais, ou use nano e cole as chaves e valores conforme `.env.example`:

```
cp .env.example .env  
  
ou  
  
nano .env
```

3 - Inicie os Contêineres em Background: O comando irá realizar o build do `.jar` via Maven dentro do Docker, construir a imagem final (com usuário não-root) e iniciar o Banco de Dados Oracle.

```
docker compose up -d --build
```

4 - Acompanhe os logs (Opcional):

```
docker compose logs -f
```

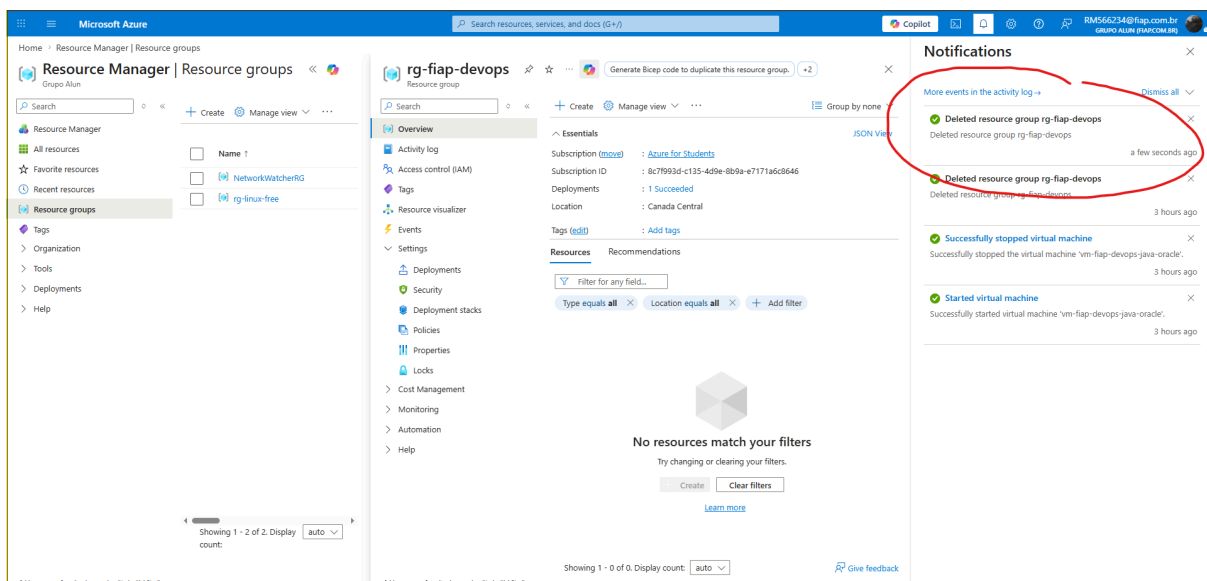
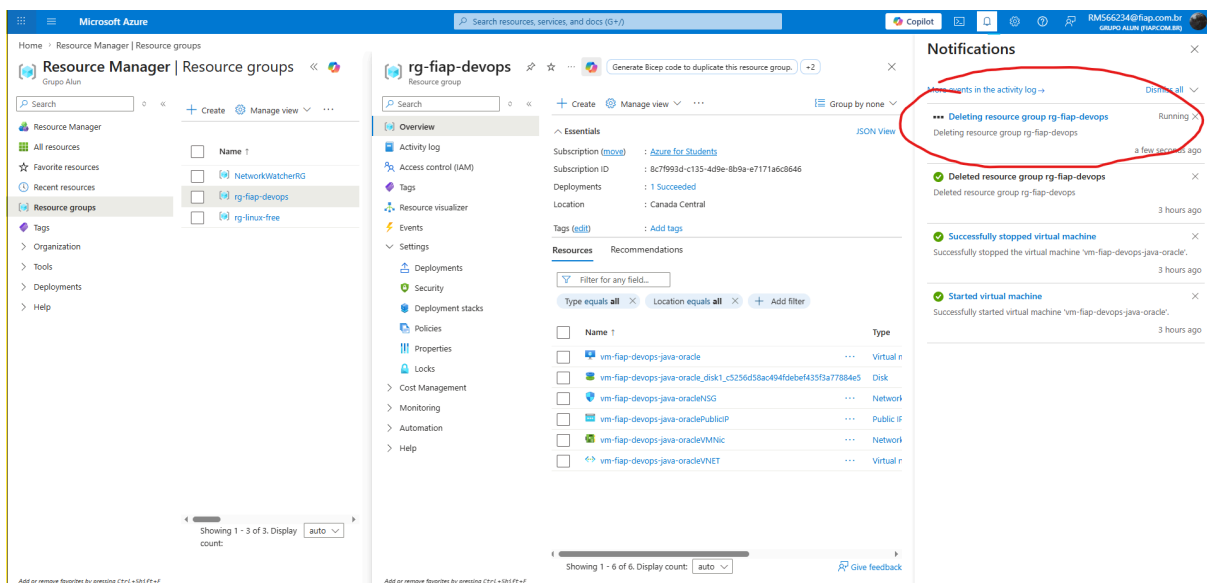
- 1.
2. **Acesse a API:** Aguarde alguns instantes até que o banco de dados inicialize completamente. O status da aplicação e a documentação interativa estarão disponíveis em:
 - **Swagger Local (caso tenha um browser):**
`http://localhost:8080/swagger-ui.html`

- **Swagger na Nuvem:**

`http://<IP_PUBLICO_DA_SUA_VM>:8080/swagger-ui.html`

Teste pela Nuvem: Se você provisionou o ambiente na Azure utilizando o nosso script `provision_azure.sh`, a porta 8080 já está aberta no NSG. Você pode testar e fazer requisições (Curl, Postman, Insomnia ou via Swagger) utilizando diretamente o IP Público da sua Máquina Virtual, demonstrando o funcionamento real em Cloud!

6. Evidência de Remoção da VM



Microsoft Azure

Search resources, services, and docs (G+)

Copilot

RM566234@fap.com.br
GRUPO ALUMI (FAPCOM.BR)

Home > Resource Manager

Resource Manager | Resource groups

Grupo Alumi

Search

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

Create Manage view Refresh Export to CSV Open query Assign tags Add to service group

Filter for any field...

Subscription equals all Location equals all Add filter

Name	Subscription	Location
NetworkWatcherRG	Azure for Students	Canada Central
rg-linux-free	Azure for Students	South Africa North

Showing 1 - 2 of 2. Display count: auto

Give feedback

7. Link do Repositório GitHub

<https://github.com/vitalis-sa/challenge1sem2026-devops>

8. Link do Vídeo no YouTube

https://www.youtube.com/watch?v=N18_M9LI8IE